プログラミング言語実験-第 3 回-

2412747　江尾祐哉

1 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai3-1.cpp -o ./2412747_kadai3-1」とコンパイルし、
「./2412747_kadai3-1 N」のようにして、N に数値を入力して実行する。

・実装概要

この課題は素数判定の課題であり、実装方法は課題 PDF にて指示されているので、それに則って実装した。

入力された整数に対してその整数以下の素数を全て求めるのだが、まずはバリデーションを行わなければならない。実行方法は引数に 1 つ値を入力する方式なので、引数が 1 つでない場合は全て弾くようにした。また、入力値は整数、それも 0 以上の整数である必要があるので、is_valid_arg 関数で判定し、bool 値によって弾くようにした。なお、ここで is_valid_arg 関数であるが、

```
while(isspace(*str))        str++;

while(*str == '+')          str++;
```

という 2 種類の while 文を用意している。これは atoi 関数が前半の空白を isspace 関数によってスキップし、直後の符号を数に関わらずスキップ(正確には負符号の数を計算)するという仕様があり、それに合わせて実装している。しかし特段やる必要はない。そのあとは数字のみの入力かを判別して、それによって true と false を返している。

上記のチェック後は atoi により引数の数字列を数値へと変換している。ここで、is_valid_arg 関数は 0 と 1 を許容しているが、素数は 2 以上であるのでこれら 2 つは不適当である。そのため、この 2 値に関しては別途エラー処理をし、異なるエラーメッセージを出すようにしている。

これらのエラー処理を終えたのち、ようやく素数判定に入る。課題 PDF に書かれて

いる方法をまとめると、

1. 大きさが N+1 の文字列型の配列を用意
2. 各インデックスに”1”を代入
3. 素数判定後、素数でないものは”1”→”0”とする

という流れで行うことになっている。したがって、まず N+1 の string 型の配列の nums を用意し、for 文で各要素に 1 を代入した。そしてその次に再度 for 文で各インデックスに対して素数判定ロジックである、is_prime 関数を適用し素数判定を行った。そうして最後に”1”の要素が入っているものを出力し、”,”と文末の改行により体裁を整えた。

素数判定のロジックには is_prime 関数を使用しているが、ここでどのようにしてロジックを組み立てたかを説明する。与えられた整数に対して判定を行うのだが、一番容易なロジックとして、2→3→4→…と整数を割る値を 1 ずつ増やして計算していく、いわゆる全探索である。しかしこれは値が大きくなったときに莫大な計算回数を要し、なおかつ今回のロジックでは N 以下の整数について全てに素数判定を行うので、計算量が$O(N^2)$となり効率が悪い。そこで、まず削れる工程だと判断したのが、割る値を 1 ずつ増やす部分である。というのも、4、6、8、…は全て偶数、すなわち 2 の倍数であるので、最初に行った 2 で割れるかの判定以降で意味をなさない。したがって最初に 2 で割り、次に 3 で、5→7→9→…というように奇数で割るようになってから割る値を 2 ずつ増やすようにした。これにより計算回数は先ほどの 2 分の 1 となる。しかし、まだ効率が悪い部分があると考えた。それは、Nが合成数であれば 2 以上$\sqrt{N}$以下の約数が存在するという性質に基づくものである。すなわち、Nが素数であるかを判定するには 2 から$\sqrt{N}$までの値で割り切れるかどうかを調べれば良いということであるので、これより割る値をiとして、$i \times i > N$となったら判定を終了すれば良いとわかる。これより、全体の計算量が$O(N\sqrt{N})$となり、劇的に効率が良くなる。

以上の点を踏まえて、まず 0 と 1 は素数ではないので不適当とし、次に 2 で割り判定し、その次に 3 から 2 ずつ増やして判定していき、$i \times i > N$となったら判定を終了するというロジックを組み立てた。もちろん余りが 0 となった瞬間に判定を終了しているので、細かい計算回数も抑えられている。なお、入力値が 2 であった場合のみ、2 は素数であるため、例外処理を行なっている。これより、素数判定が可能になり、この課題の要件を満たせた。

・実行結果

```
e2412747@a14:~/project/cpj2026_2412747_3$ ./2412747_kadai3-1 57
2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53
e2412747@a14:~/project/cpj2026_2412747_3$ ./2412747_kadai3-1 101
2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,83,89,97,101
e2412747@a14:~/project/cpj2026_2412747_3$ ./2412747_kadai3-1 1949
2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,83,89,97,101,103,1
07,109,113,127,131,137,139,149,151,157,163,167,173,179,181,191,193,197,199,211,2
23,227,229,233,239,241,251,257,263,269,271,277,281,283,293,307,311,313,317,331,3
37,347,349,353,359,367,373,379,383,389,397,401,409,419,421,431,433,439,443,449,4
57,461,463,467,479,487,491,499,503,509,521,523,541,547,557,563,569,571,577,587,5
93,599,601,607,613,617,619,631,641,643,647,653,659,661,673,677,683,691,701,709,7
19,727,733,739,743,751,757,761,769,773,787,797,809,811,821,823,827,829,839,853,8
57,859,863,877,881,883,887,907,911,919,929,937,941,947,953,967,971,977,983,991,9
97,1009,1013,1019,1021,1031,1033,1039,1049,1051,1061,1063,1069,1087,1091,1093,10
97,1103,1109,1117,1123,1129,1151,1153,1163,1171,1181,1187,1193,1201,1213,1217,12
23,1229,1231,1237,1249,1259,1277,1279,1283,1289,1291,1297,1301,1303,1307,1319,13
21,1327,1361,1367,1373,1381,1399,1409,1423,1427,1429,1433,1439,1447,1451,1453,14
59,1471,1481,1483,1487,1489,1493,1499,1511,1523,1531,1543,1549,1553,1559,1567,15
71,1579,1583,1597,1601,1607,1609,1613,1619,1621,1627,1637,1657,1663,1667,1669,16
93,1697,1699,1709,1721,1723,1733,1741,1747,1753,1759,1777,1783,1787,1789,1801,18
11,1823,1831,1847,1861,1867,1871,1873,1877,1879,1889,1901,1907,1913,1931,1933,19
49
e2412747@a14:~/project/cpj2026_2412747_3$ ./2412747_kadai3-1 0
0と1は素数ではありません。2以上の整数を入力してください
e2412747@a14:~/project/cpj2026_2412747_3$ ./2412747_kadai3-1 1
0と1は素数ではありません。2以上の整数を入力してください
e2412747@a14:~/project/cpj2026_2412747_3$ ./2412747_kadai3-1 -46
数値の入力が不正です。2以上の整数を入力してください
e2412747@a14:~/project/cpj2026_2412747_3$ ./2412747_kadai3-1
引数を1つにして実行してください
e2412747@a14:~/project/cpj2026_2412747_3$ ./2412747_kadai3-1 abc
数値の入力が不正です。2以上の整数を入力してください
e2412747@a14:~/project/cpj2026_2412747_3$ ./2412747_kadai3-1 2
2
```

図1　課題1の実行結果

2 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai3-2.cpp -o ./2412747_kadai3-2」とコンパイルし、
「./2412747_kadai3-2 integer2026.txt」のようにして、ファイルを指定して実行する。

・実装概要

この課題はファイル内に記載されている複数の整数を昇順にソートするというものであり、またソート方法が課題 PDF によって指定されている。そのため、まずはファイルを開き、そこに記載されている整数を配列に保持してあげる必要がある。また、要素数

の変化によらず実行できる必要がある。なお、今回のプログラムはファイルの内容が妥当である前提で動いている。これは色々なバリデーションを追加すると、その分コードが複雑になってしまうからである。

要素数が変わっても実行できるようにするためには、ファイル内にある整数を数える必要があると考えられる。そのため、まずは calc_num_of_nums_in_file 関数で要素数の計算を行った。要素数をカウントする int 型の count と、ファイルの要素を読み込む int 型の tmp を用意し、fin >> tmp の回数に応じて count をインクリメントしている。これは fin>>tmp が空白や改行ごとによって要素を分割することを利用している処理である。これにより要素数をカウントすることができるので、これを main に返している。なお、この関数内でファイルをクローズしているが、これは堅牢なプロジェクトにするために行なっている。ただ、ファイルの要素を取り込むためにもう一度ファイルを開く必要があるのが面倒であるが、スコープがめちゃくちゃになるのを防ぎたいため、このような処理をしている。

これで要素数を求められたので、この大きさの int 型の配列を用意し、再度ファイルを開いて配列に整数を格納していった。これにより配列する対象となるデータを構成できた。これより、この配列の要素を昇順ソートして、あとは順番に出力すれば良い。

昇順ソートのやり方は PDF で指示されており、

1. 配列の 0 からn−1 の要素について最大値を探す。
2. 最大値を一番後ろの値と交換する。
3. ソート対象のデータを 1 つ減らし、0 番目からn−2 番目について最大値を探す。以降、1 から 3 を繰り返す。

という方法である。これは選択ソートであるが、通常選択ソートは最小値を寄せてソートしていく例が多く、最大値を寄せるという動作はバブルソートに近いと感じた。このソート方法を int の配列に適用することで、無事整列されることとなる。ここで、値の交換にはポインタを用いた自作関数を使用しなければならないと課題 PDF に記載されているので、値の変更には ft_swap 関数という自作関数を用いた。この関数は引数に 2 つの int ポインタを持ち、関数内部で int 型の tmp 変数という値の仮置きの変数を用いて、引数に与えられた変数の値を交換している関数となっている。ポインタを用いているので関数内部で値を変更しても、その変更が外側でも適用されることとなっている。

これらの実装により、integer2026.txt に記載されている整列を昇順でソートすることが可能となった。

・実行結果

```
e2412747@a14:~/project/cpj2026_2412747_3$ ./2412747_kadai3-2 integer2026.txt
4,5,6,7,8,10,13,14,17,21,22,24,25,26,27,28,29,31,32,33,35,38,40,41,43,44,45,48,5
0,53,54,55,56,57,59,63,64,65,66,68,69,70,73,74,78,79,82,83,84,85,86,87,88,90,91,
95,96,98,99,101,102,103,104,105,107,108,112,114,117,118,119,120,121,123,124,128,
129,132,134,135,136,137,138,140,141,142,144,145,147,152,153,154,159,162,164,165,
166,170,173,175,176,177,181,184,186,190,191,192,193,195,199,201,202,204,205,206,
212,213,216,217,218,219,222,223,225,228,229,231,235,236,238,239,242,246,247,248,
249,252,253,254,258,259,260,262,264,265,267,271,279,281,283,289,290,291,293,294,
295,303,304,305,309,311,312,316,317,321,322,323,324,326,328,331,332,334,337,339,
340,342,345,348,350,351,354,357,358,359,361,364,366,370,371,372,374,375,377,379,
381,384,385,388,391,392,397,402,403,407,410,411,415,416,420,421,423,425,430,431,
434,436,437,438,439,443,444,446,447,448,449,451,453,454,456,458,459,460,461,462,
464,465,466,467,468,469,470,472,473,474,475,476,477,478,479,480,481,483,485,487,
488,496,498,500,501,503,504,505,511,513,514,515,517,518,519,520,522,523,526,531,
532,534,536,537,538,540,541,543,544,546,551,554,558,561,565,566,567,570,571,576,
581,582,588,589,594,596,597,598,600,602,604,607,614,615,616,617,618,624,625,627,
628,629,630,631,632,635,637,640,644,645,646,649,650,654,655,656,657,658,661,663,
666,672,675,676,677,678,679,680,682,683,685,688,689,690,691,696,698,699,700,701,
703,706,709,710,711,712,719,720,721,722,723,725,727,728,729,730,732,733,735,737,
742,749,752,754,755,762,764,765,767,772,775,776,777,778,779,780,781,782,783,785,
786,788,792,795,796,798,799,802,803,809,813,816,817,818,819,820,821,826,827,830,
834,837,839,841,842,844,845,846,847,853,855,856,857,858,859,860,862,864,871,873,
875,877,879,881,884,885,887,888,891,892,896,901,902,904,906,907,910,913,915,917,
922,923,926,927,932,933,934,935,937,942,943,944,945,946,947,948,949,951,955,956,
958,959,962,963,964,966,967,968,969,970,973,975,978,979,983,984,986,990,994,996,
997,998,999,1000
```

図2　課題2の実行結果

3 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai3-3.cpp -o ./2412747_kadai3-3」とコンパイルし、
「./2412747_kadai3-3」と実行する。

・実装概要

数値が代入された変数を uint8_t ポインタにキャストし、これにより値の格納方法がリトルエンディアンかビッグエンディアンかを判別するプログラムの実装をしなければならないが、ここで一度リトルエンディアンがどういうものかについてまとめる。

エンディアンとは、マルチバイトのデータをメモリ上に並べる順番の規則のことである。また、マルチバイトとは、コンピュータが一度に扱うデータの基本単位であるバイト

での 1 バイトで足りない大きな値を扱うために、複数のバイトで表すことを指す。エンディアンにはリトルエンディアンとビッグエンディアンの 2 種類が存在し、前者は下位バイトが先に、後者は上位バイトが先に並ぶようになっている。例えば 4 バイトの 0x12345678 を仮定したときに、先頭から末尾にかけてビッグエンディアンは、

12 34 56 78

とメモリ上に並び、対してリトルエンディアンは、

78 56 34 12

というようにメモリ上に並ぶようになっている。これはコンピュータの CPU 設計によって分かれており、ビッグエンディアンは可読性が高いことやネットワークプロトコルとの相性が良いことがメリットとして挙げられ、リトルエンディアンは下位ビットの計算効率がよく、小さい型へのキャストがしやすいことがメリットである。

これでこの課題の概要は説明できたため、実装の説明を行う。数値を代入された変数を uint8_t ポインタへキャストする理由は、uint8_t は 1 バイトをそのまま扱える型であり、メモリはバイト単位で連続していることから、uint8_t ポインタへキャストすることでその変数の中身を 1 バイトずつ確認できるからである。このキャストによってリトルエンディアンかの判別が可能となる。本コードでは int 型の num に 1 を代入し、それを uint8_t のポインタへキャストして if 分岐によってリトルエンディアンかどうかを判別している。もし仮にリトルエンディアンであれば、

01 00 00 00

となり、ビッグエンディアンであれば、

00 00 00 01

となる。そして、if(*(uint8_t *)&num == 1)の条件によって先頭のバイトを読むため、もしリトルエンディアンであれば 0 となり、ビッグエンディアンであれば 1 となる。これによってリトルエンディアンかが判別でき、これはマシン依存であるので実装した check_endian 関数を実行さえすればプログラムは完了となる。ただし、num の値を 0 とすると両者の区別ができなくなるため、その点には注意する必要がある。

・実行結果

```
e2412747@a14:~/project/cpj2026_2412747_3$ ./2412747_kadai3-3
リトルエンディアンです。
```

図3　課題3の実行結果

4 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai3-4.cpp -o ./2412747_kadai3-4」とコンパイルし、「./2412747_kadai3-4 fig2026.ppm」というように、引数に ppm の画像を指定して実行する。

・実装概要

この課題は、PPM フォーマットの画像の読み込み、グレースケール画像への変換、フィルターの適用の 3 ステップに分かれている。したがってこの 3 つの工程に沿って順番に説明していく。

まず、PPM とは 8bit 以上の階調を扱うことができ、最も簡単にプログラミングできる非圧縮の画像フォーマットの PNM(Portable aNyMap)の 1 種である。PNM では、画像情報をテキストもしくはバイナリでもつのだが、それらに関係なく必ず始めにテキスト形式のヘッダを持っている。1 行目には PNM の種類を示すマジックナンバーが P1 から P6 のどれかで書かれており、2 行目には幅と高さ、3 行目には最大輝度について書かれている。今回使用するファイルは P6 のフルカラー画像をバイナリ形式であり、指示はされていないが、出力するグレースケールの画像は P2 のテキスト形式とした。これらを踏まえてコードを追って説明していく。

引数のエラー処理を終えたあとファイルを開くが、このとき ios::binary と指定することでファイルをバイナリ形式で開くことができる。これは basic_ifstream クラスのコンストラクタの引数として、std::ios_base::openmode というものがあり、そこの openmode に binary を指定しているということである。これによりファイルが開けたら、次は画像の縦と横の変数を宣言し、アドレスを file_check 関数に渡している。この関数ではヘッダの読み取りを順番に行い、縦幅と横幅は引数の int ポインタの値に代入している。また、今回は

ppm を表す P1 という文字列は使用せず、また最大画素値の 255 も、それを前提としたロジックでプログラムしているために使用しないので、void でキャストしてこれらの変数を利用しないことを明示している。最後に ifstream の親クラスの istream のメンバ関数である get を使用して改行を一つ読み飛ばすことで、次のバイナリの読み取りにスムーズに移れるようにした。

次に画像の情報を読み取っていくのだが、その前にそれを格納するための 2 次元配列の window を定義し、これに new 演算子を用いて動的にメモリを確保させた。確保させるサイズを指定して new によりメモリを確保するのだが、new によるメモリ確保が失敗した時に正常に終了するための処理を施した。というのもあまり発生することはないが、new によるメモリの確保の失敗は起こりうるものであり、有限リソースを扱う以上、正常でない動作が起こった場合には安全にプログラムを終了させる必要があるからである。通常 new は失敗すると std::bad_alloc の例外を投げる。これに対処するために try-catch 文を使用してコードを書くのだが、C 言語での malloc のエラー処理の記述の方が慣れているため、new に std::nothrow_t という型を指定してあげて、失敗時には例外を出さずに null ポインタを返させるようにしている。これにより自身の馴染み深いエラー処理を実現させており、メモリの確保が失敗した際にはエラーメッセージを出してプログラムを異常終了させるようにした。2 次元配列のメモリ確保後はそれぞれのインデックスについて 1 次元配列の動的メモリを確保した。ここでもメモリエラーについて考えており、この場合は途中で失敗した時を想定して、今までに確保したメモリを安全に全て解放させるような delete_window 関数を実装して適切な処置を施した。

上記のメモリ確保が終了したら、その変数を get_gray_ppm 関数に渡し、画像の各点に対して値を格納していった。この関数では、各点を 1 行ごと順番に、読み取った画素情報をグレースケールの演算式を通して格納している。今回の最大画素値は 255 であり、画素値は 0～255 で表されるので、型範囲の一致している unsigned char 型を用いて画素値を読み取っている。ただし読み取りの際に実行する、ifstream のメンバ関数の read は引数の型が char であるので、明示的にキャストして使用している。1 ピクセルは 3 バイトであるから、ここで 3 バイト読み取るよう指定することで unsigned char 配列にそれぞれ赤、緑、青の値が入ることとなっている。そしてこれらの値を基に、グレースケールの変換式として一般的に知られている、$r \times 0.299 + g \times 0.587 + b \times 0.114$により、各ピクセルに対するグレースケールの値を求めることができるので、これを各点に代入して書き換えた。このとき、get_gray_ppm ではファイルの読み込みを行なっており、もちろんファイルの読み取りが失敗する可能性も考えられるため、この関数の返り値を bool 型として、false が返ったときは、main 関数で既に確保したメモリ

を解放し、ファイルを閉じてエラー文とともに異常終了させるようにした。正常に関数が実行された際にももうこれ以上ファイルを扱うことはないので、main 関数でファイルを閉じるようにした。

次にグレースケールに対してフィルターを適用させたものを用意する必要があるので、新たに int 型の 2 次元配列を用意した。これも window のときと同様にメモリの動的確保とエラー対策を施した。その後に apply_filter 関数によって、先ほど保存したピクセルの情報をもとにフィルターを適用させた。フィルターの適用の方法は課題 PDF に記載されており、それはある点を中心として隣同士の点は等倍、ある点自身は-4 倍してそれぞれ足し合わせるという方法である。これにより各点について行い、最大画素値が 255 であるため、値が 255 を超えた際には 255 にするという処理を行い、各ピクセルについて求めた。しかしこのフィルターは画像の端のピクセルには適用ができないことに留意する必要がある。ただし、先ほどの filter のメモリ確保において、

```
filter[j] = new(nothrow) int[img_width]();
```

と最後に()をつけることで 0 埋めをすることができ、これにより filter にゴミ値が入った状態のまま画像が出力されるということを防ぐことができる。このような処理をすることでフィルターが適用されない場所についても考えた。この関数の実行後、始めに確保した window2 次元配列は不要となるため、delete_window 関数によってメモリの解放を行なった。

最後に filter の各点の情報をファイルに書き出し pgm 画像を出力させるために、save_pgm 関数というものを実装した。この関数では、まず出力用ストリームの fout を用意し、次にヘッダを書き込み、最後にテキスト形式で各点の画素値を書き込んでファイルを出力させている。PNM の形式に沿って、今回出力するファイルはテキスト形式のグレースケールであるため、マジックナンバーは P2 となる。その後縦幅と横幅を入力し、最大画素値の 255 を入力する。そして各点の情報を空白と改行を挟んで、1 行ずつ書き込んでいる。もちろん生成エラーにも対応させている。これにより全ての工程が終了したら、最後に output.pgm が出力されたことをメッセージで示して、使用したリソースを全て解放して終了となる。

・出力結果

```
e2412747@a14:~/project/cpj2026_2412747_3$ ./2412747_kadai3-4 fig2026.ppm
output.pgmに保存しました。
```

図4-1　課題4の実行結果

図4-2　課題4の出力結果

5 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai3-5.cpp -o ./2412747_kadai3-5」とコンパイルし、「./2412747_kadai3-5 enrollment2026.txt input2026.txt」というように、第 1 引数に 300 人分のテンプレート作成用の顔データの載っているテキストファイル、第 2 指定して実行する。

・実装概要

バイオメトリクスとは、顔認証や指紋認証、そして第 1 回の課題 5 で取り扱った虹彩認証などの、人間の身体の特徴を用いて人物を特定する生体認証のことである。この課題では簡易的な顔認証システムの作成を行い、データとして一人あたり 5 枚の顔画像、それを 300 人分与えられる。各人の 5 枚の画像の情報を平均化することで、「平均顔」と呼ばれる、その人を代表する画像データを作成し、それらのデータを基に、入力された任意のデータから、それが誰の顔画像であるかを判別するという仕組みである。このようなプログラムで成り立っているので、いわばこの顔認証システムは機械学習の基礎であると考えられる。このように前提が何であるかをまとめ、これを踏まえて実装の説明をすることで、全体の解像度が上がると考えた。

課題 PDF には(1)～(3)が書かれているので、まずは(1)について説明する。ここでは第一引数の enrollment2026.txt を開き、用意した平均顔用 2 次元配列の avg にファイルから読み取った値を格納していく。ここで今回は vector の使用が不可であるため、値を格納していく変数は静的もしくは動的にメモリを確保されたものを用いなければならない。スタック領域のメモリ領域はヒープに比べてかなり小さいため、今回のような 1000 次元ものデータを扱うプログラムでは不向きである。この事実があるため当初は動的確保によりコード内の変数のメモリを確保していたが、このプログラム全体では定義する配列の数が多く、メモリのエラー処理を含めてコードを記述するとものすごくコードが複雑になり実装した内容の整理が非常に難しかった。そのため今回は avg と後に使う test_data はグローバル変数として定義し、それを用いることにした。そのほかについては静的な配列とした。

脱線したが、テキストファイルには 1 人につき 5 個のデータがあるのでそれを足し合わせ、そして 5 でわることで平均値の算出を行った。これにより平均顔データが作成できる。ここで読み取りについては read_vector 関数を実装し、それを用いた。この関数は、今回使用するテキストファイルの記述形式に合わせて値を読み取る関数となっている。また、(2)のフェイズにおいて、input2026.txt の値読み取りにも使用するため、汎用性のあるものとしている。まず文字列型の line で getline 関数を使用して 1 行ずつ抜き出していく。ファイルには ID のみ記載の行と、カンマ区切りで値が記されている行があるので、if 分岐で string::npos のメンバ定数を用いて、カンマが存在しなければ工程を飛ばすようにした。次にカンマで区切られた文字列を解析していく必要があるため、それを可能とする stringstream を使用し、string 型の token で字句解析により分割された文字列を格納した。1 行あたり 1000 個のデータがあるので、その回数分文字列を値に変換して足し合わせて、規定回数行うと終了するようにした。これにより(1)の工程が完了となる。

次に(2)について、ここでは先ほど作成した平均顔データを用いて入力された顔データとの比較を行う。Input2026.txt は、enrollment2026.txt に対応した順番で 300 人のデータが記載されている。そのため各人に対して正解がどの位置にあるかを記録する配列を用意した。これを用いてまずは read_vector 関数で値を格納し、次に double の配列の dists で 1 人のデータを 300 人の平均顔のデータとのユークリッド距離を計算し、それの 2 乗値を加算していく。これで、例えば 20 番目の顔データについて、20 番目の平均顔データよりも精度の良い比較、すなわちユークリッド距離の 2 乗値の小さいものがあれば、その数を記録していくことにする。これを rank として、定義した rank_list の各要素に格納していく。これを行うことで識別率がいくらであるかの計算を可能とした。なお、順位に対応させるために初期値を 1 と設定した。以上より、correct_rank 変数に rank が 1 であった数を入れて、これを人数で割ることで識別率の算出が完了となった。識別率の出力であるが、fixed << setpresion(2)を行うことで、double の値の出力を小数第 2 位で固定させた。

最後に(3)についてである。ここでは(2)で識別率を求めた手順で、rank が 1 まで、2 まで、3、…というように拡張して、識別率がいくらであるかを算出している。したがってやることは対して変わらず、先ほど作成した rank_list 配列について、計算していく rank の値を 1 ずつ増やして全探索していくだけである。この値を用いてグラフ化する必要があるため、今回は csv ファイルに小数第 2 位までを指定して出力させた。この csv ファイルからエクセルでグラフを作成し、それを下記に載せた。以上より、この課題の実装説明が完了した。

・実行結果

```
e2412747@a14:~/project/cpj2026_2412747_3$ ./2412747_kadai3-5 enrollment2026.txt
input2026.txt
1位識別率: 25.67 %
累積識別率データをresult.csvに出力しました。
```

図5-1　課題5の実行結果

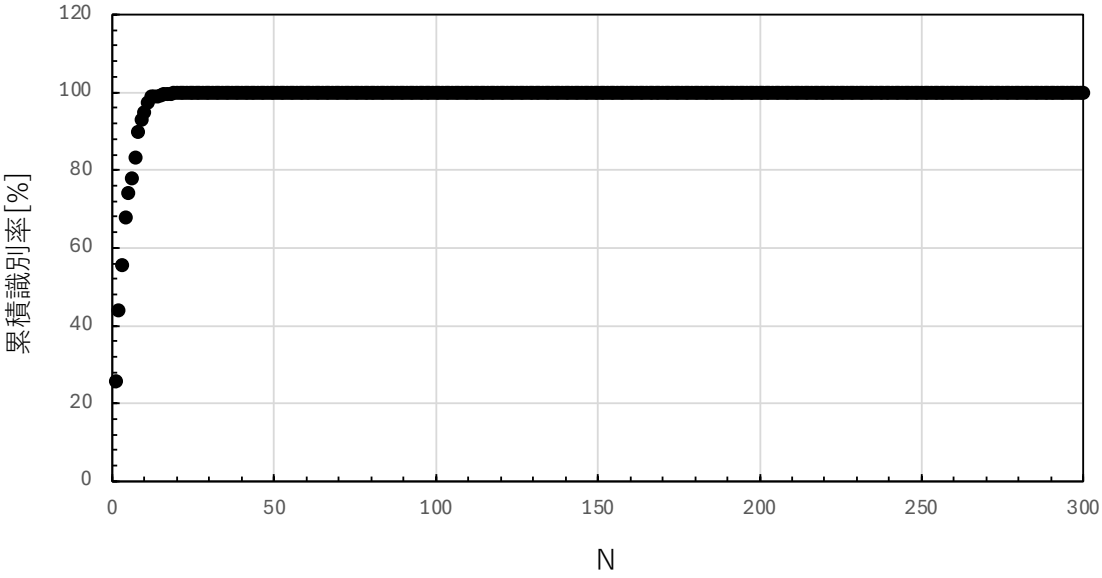

図5-2　累積識別率のグラフ